Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

# Звіт

## З ЛАБОРАТОРНОЇ РОБОТИ № 1

з дисципліни «Професійний Python»
на тему: «Організація професійного Python-проєкту та Git»

**Виконала:**
студентка 3 курсу
групи ФеП-33
Наборська Дарія
Русланівна

**Перевірив:**
професор Свелеба Сергій
Андрійович

Львів-2026

## Мета роботи

Ознайомитися з основними принципами організації професійного Python-проєкту, навчитися створювати ізольоване середовище розробки, формувати структуру проєкту за принципом src-layout, працювати з модулями та пакетами Python, створювати файл pyproject.toml, використовувати Git для контролю версій та оформлювати базову документацію проєкту.

## Номер індивідуального варіанта

Варіант 9. Облік спортивних результатів.

## Постановка задачі

Створити консольний застосунок для обліку спортивних результатів. Для сутності Athlete (спортсмен) зберігати:

- ім'я;
- вид спорту;
- результат (у балах);
- категорію.

Реалізувати функції:

- додавання результату;
- пошук за видом спорту;
- визначення найкращого результату;
- обчислення середнього результату;
- формування рейтингу.

Проєкт має використовувати src-layout, типізацію (type hints), щонайменше три модулі, точку входу з конструкцією if __name__ == "__main__":, містити конфігурацію pyproject.toml, .gitignore, README.md та історію комітів у Git.

**Завдання підвищеної складності**

Додатково реалізовано такі можливості з переліку завдань підвищеної складності:

- **Menu-driven interface з введенням даних із клавіатури** — програма працює в циклі через консольне меню з 7 командами, усі дані нового спортсмена вводяться користувачем.
- **Validation input із власним exception** — модуль validators.py перевіряє введені значення (порожні рядки, цифри в імені, нечислові результати, вихід за межі [0; 100], неіснуюча категорія); у разі помилки генерується власний виняток ValidationError, а для порожнього списку — AthleteNotFoundError (exceptions.py).

Також у проєкті використано dataclass, окремий модуль configuration constants (config.py), сортування (рейтинг) та README з прикладом запуску програми.

Репозиторій проєкту: https://github.com/dashanaborska/python_lab1 (звіт — у каталозі report/).

## Структура проєкту

```
None
python_lab1/
├── .gitignore
├── pyproject.toml
├── README.md
├── report/
│   ├── Lab_01_Naborska.docx
│   └── Lab_01_Naborska.pdf
├── src/
│   └── sport_results/
│       ├── __init__.py
│       ├── config.py
│       ├── exceptions.py
│       ├── main.py
│       ├── models.py
│       ├── services.py
│       └── validators.py
└── tests/
    └── test_services.py
```

## Опис створених модулів

- **src/sport_results/__init__.py**: позначає каталог як Python-пакет і фіксує версію пакета (__version__ = "0.1.0").
- **src/sport_results/models.py**: описує модель даних Athlete за допомогою @dataclass із типізованими полями name: str, sport: str, result: float, category: str.
- **src/sport_results/services.py**: модуль бізнес-логіки з функціями add_result, find_by_sport, find_best_result, calculate_average_result, build_rating.
- **src/sport_results/validators.py**: перевірка введених із клавіатури даних (validate_text, validate_result, validate_category).
- **src/sport_results/exceptions.py**: власна ієрархія винятків SportResultsError → ValidationError, AthleteNotFoundError.
- **src/sport_results/config.py**: конфігураційні константи MIN_RESULT, MAX_RESULT, CATEGORIES, RATING_SIZE.
- **src/sport_results/main.py**: консольна точка входу застосунку. Створює початкові демонстраційні дані, виводить меню, зчитує команди користувача, викликає функції сервісного шару та виводить результат у табличному вигляді. Запуск контролюється перевіркою if __name__ == "__main__":.

- **tests/test_services.py**: автоматизовані тести pytest для бізнес-логіки та валідації.

## Текст основних фрагментів коду

### src/sport_results/models.py

```python
from dataclasses import dataclass


@dataclass
class Athlete:
    name: str
    sport: str
    result: float
    category: str
```

### src/sport_results/config.py

```python
MIN_RESULT: float = 0.0
MAX_RESULT: float = 100.0
CATEGORIES: tuple[str, ...] = (
    "МСМК",
    "МС",
    "КМС",
    "I розряд",
    "II розряд",
    "III розряд",
)
RATING_SIZE: int = 10
```

### src/sport_results/exceptions.py

```python
class SportResultsError(Exception):
    pass


class ValidationError(SportResultsError):
    pass


class AthleteNotFoundError(SportResultsError):
    pass
```

### src/sport_results/services.py

```python
from sport_results.config import RATING_SIZE
from sport_results.exceptions import AthleteNotFoundError
from sport_results.models import Athlete


def add_result(athletes: list[Athlete], athlete: Athlete) -> None:
```

```
    athletes.append(athlete)


def find_by_sport(athletes: list[Athlete], sport: str) -> list[Athlete]:
    query = sport.strip().lower()
    return [athlete for athlete in athletes if athlete.sport.lower() == query]


def find_best_result(athletes: list[Athlete]) -> Athlete:
    if not athletes:
        raise AthleteNotFoundError("Список результатів порожній.")
    return max(athletes, key=lambda athlete: athlete.result)


def calculate_average_result(athletes: list[Athlete]) -> float:
    if not athletes:
        return 0.0
    return sum(athlete.result for athlete in athletes) / len(athletes)


def build_rating(
    athletes: list[Athlete],
    limit: int = RATING_SIZE,
) -> list[tuple[int, Athlete]]:
    ordered = sorted(
        athletes,
        key=lambda athlete: (-athlete.result, athlete.name),
    )
    return list(enumerate(ordered[:limit], start=1))
```

## src/sport_results/validators.py

```
Python
from sport_results.config import CATEGORIES, MAX_RESULT, MIN_RESULT
from sport_results.exceptions import ValidationError


def validate_text(value: str, field: str) -> str:
    cleaned = value.strip()
    if not cleaned:
        raise ValidationError(f"Поле '{field}' не може бути порожнім.")
    if any(char.isdigit() for char in cleaned):
        raise ValidationError(f"Поле '{field}' не повинно містити цифр.")
    return cleaned


def validate_result(value: str) -> float:
    try:
        result = float(value.strip().replace(",", "."))
    except ValueError as error:
        raise ValidationError(f"Результат '{value}' не є числом.") from error
    if not MIN_RESULT <= result <= MAX_RESULT:
        raise ValidationError(
            f"Результат має бути в діапазоні [{MIN_RESULT}; {MAX_RESULT}]."
        )
    return result


def validate_category(value: str) -> str:
    cleaned = value.strip()
    for category in CATEGORIES:
```

```
        if category.lower() == cleaned.lower():
            return category
    raise ValidationError(
        f"Невідома категорія '{cleaned}'. Допустимі: {', '.join(CATEGORIES)}."
    )
```

### src/sport_results/main.py

```python
from collections.abc import Callable
from typing import TypeVar

from sport_results.exceptions import SportResultsError
from sport_results.models import Athlete
from sport_results.services import (
    add_result,
    build_rating,
    calculate_average_result,
    find_best_result,
    find_by_sport,
)
from sport_results.validators import (
    validate_category,
    validate_result,
    validate_text,
)

T = TypeVar("T")


def create_demo_athletes() -> list[Athlete]:
    return [
        Athlete("Олена Ковальчук", "Плавання", 87.5, "КМС"),
        Athlete("Максим Бондар", "Легка атлетика", 92.3, "МС"),
        Athlete("Ірина Мельник", "Плавання", 78.4, "І розряд"),
        Athlete("Андрій Шевченко", "Бокс", 81.0, "КМС"),
        Athlete("Софія Ткачук", "Легка атлетика", 95.1, "МСМК"),
    ]


def print_athletes(athletes: list[Athlete]) -> None:
    if not athletes:
        print("Записів не знайдено.")
        return
    print(
        "Ім'я".ljust(20)
        + "Вид спорту".ljust(18)
        + "Результат".rjust(10)
        + "  Категорія"
    )
    print("-" * 62)
    for athlete in athletes:
        print(
            f"{athlete.name:20}"
            f"{athlete.sport:18}"
            f"{athlete.result:10.2f}  "
            f"{athlete.category}"
        )


def print_rating(athletes: list[Athlete]) -> None:
```

```
    print(
        "Місце".ljust(7)
        + "Ім'я".ljust(20)
        + "Вид спорту".ljust(18)
        + "Результат".rjust(10)
    )
    print("-" * 55)
    for place, athlete in build_rating(athletes):
        print(
            f"{place:<7}"
            f"{athlete.name:20}"
            f"{athlete.sport:18}"
            f"{athlete.result:10.2f}"
        )


def ask(prompt: str, validator: Callable[[str], T]) -> T:
    while True:
        try:
            return validator(input(prompt))
        except SportResultsError as error:
            print(f"Помилка: {error}")


def validate_name(value: str) -> str:
    return validate_text(value, "ім'я")


def validate_sport(value: str) -> str:
    return validate_text(value, "вид спорту")


def input_athlete() -> Athlete:
    return Athlete(
        name=ask("Ім'я спортсмена: ", validate_name),
        sport=ask("Вид спорту: ", validate_sport),
        result=ask("Результат (бали): ", validate_result),
        category=ask("Категорія: ", validate_category),
    )


def print_menu() -> None:
    print()
    print("===== Облік спортивних результатів =====")
    print("1. Показати всі результати")
    print("2. Додати результат")
    print("3. Пошук за видом спорту")
    print("4. Найкращий результат")
    print("5. Середній результат")
    print("6. Рейтинг спортсменів")
    print("0. Вихід")


def run_menu(athletes: list[Athlete]) -> None:
    while True:
        print_menu()
        command = input("Оберіть команду: ").strip()
        print()
        try:
            if command == "1":
                print_athletes(athletes)
            elif command == "2":
```

```
                athlete = input_athlete()
                add_result(athletes, athlete)
                print(f"Додано: {athlete.name} ({athlete.result:.2f})")
            elif command == "3":
                sport = ask("Вид спорту: ", validate_sport)
                print_athletes(find_by_sport(athletes, sport))
            elif command == "4":
                best = find_best_result(athletes)
                print(
                    f"Найкращий результат: {best.name} — "
                    f"{best.sport}, {best.result:.2f} ({best.category})"
                )
            elif command == "5":
                average = calculate_average_result(athletes)
                print(f"Середній результат: {average:.2f}")
            elif command == "6":
                print_rating(athletes)
            elif command == "0":
                print("До побачення!")
                break
            else:
                print("Невідома команда.")
        except SportResultsError as error:
            print(f"Помилка: {error}")


def main() -> None:
    athletes = create_demo_athletes()
    run_menu(athletes)


if __name__ == "__main__":
    main()
```

## Пояснення pyproject.toml

```toml
[build-system]
requires = ["setuptools>=70"]
build-backend = "setuptools.build_meta"

[project]
name = "sport-results"
version = "0.1.0"
description = "Sport results accounting laboratory project"
readme = "README.md"
requires-python = ">=3.11"
authors = [{ name = "Daria Naborska" }]

[project.scripts]
sport-results = "sport_results.main:main"

[tool.setuptools.packages.find]
where = ["src"]
```

Файл стандарту PEP 518/621, що описує конфігурацію збирання проєкту:

- **[build-system]**: використовує setuptools як рушій збирання (build-backend = "setuptools.build_meta").
- **[project]**: задає метадані пакета — ім'я sport-results, версію 0.1.0, опис, файл README, автора та вимогу до інтерпретатора requires-python = ">=3.11".
- **[project.scripts]**: реєструє консольну команду sport-results = "sport_results.main:main".
- **[tool.setuptools.packages.find]**: налаштовує пошук пакетів у каталозі src/ для editable installation (python -m pip install -e .).

## Вміст .gitignore

Файл виключає з Git virtual environment, кеш, файли IDE, артефакти збирання та секрети:

```
None
.venv/
__pycache__/
*.pyc
.pytest_cache/
.idea/
.vscode/
dist/
build/
*.egg-info/
.env
```

## Фрагмент README.md

```
Markdown
# Sport Results

Лабораторна робота №1 з дисципліни «Професійний Python».
Варіант 9 — «Облік спортивних результатів».

## Опис

Консольний застосунок для обліку результатів спортсменів.
Для кожного спортсмена зберігаються ім'я, вид спорту, результат (у балах) та
категорія.

Можливості:

- додавання результату;
- пошук за видом спорту;
- визначення найкращого результату;
- обчислення середнього результату;
- формування рейтингу спортсменів.

Завдання підвищеної складності:

- menu-driven interface з введенням даних із клавіатури;
- validation input із власними винятками (`ValidationError`,
`AthleteNotFoundError`);
- окремий модуль configuration constants (`config.py`);
```

````
- використання `dataclass`;
- сортування (рейтинг);
- README з прикладом запуску програми.

## Requirements

Python 3.11+

## Installation

```bash
python -m venv .venv
source .venv/bin/activate        # Linux/macOS
.venv\Scripts\activate           # Windows
python -m pip install -e .
```

## Run

```bash
python -m sport_results.main
```

або

```bash
sport-results
```
````

# Screenshots запуску application

## Встановлення пакета у virtual environment (editable mode)

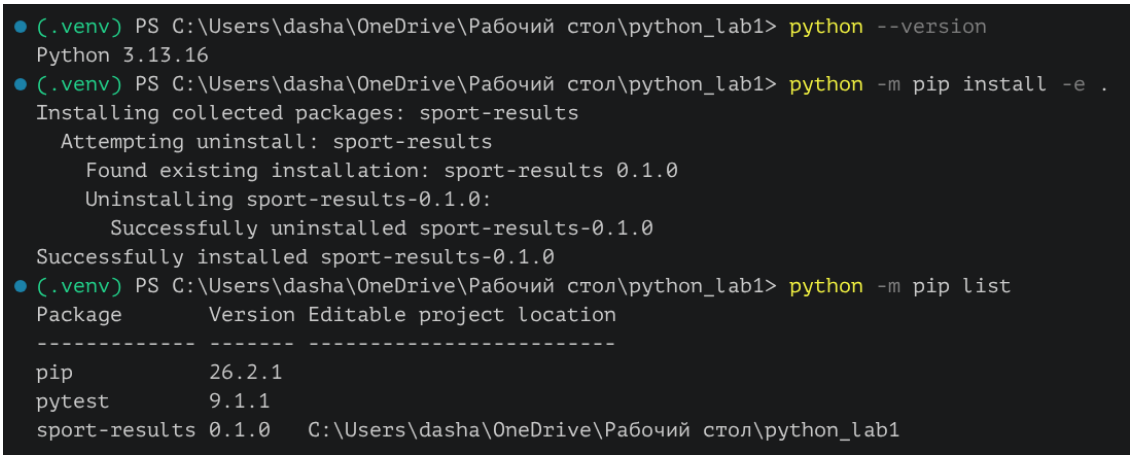



*Рисунок 1 — Перевірка Python та встановлення пакета*

## Перегляд списку, додавання результату та валідація введення

```
(.venv) PS C:\Users\dasha\OneDrive\Рабочий стол\python_lab1> sport-results

===== Облік спортивних результатів =====
1. Показати всі результати
2. Додати результат
3. Пошук за видом спорту
4. Найкращий результат
5. Середній результат
6. Рейтинг спортсменів
0. Вихід
Оберіть команду: 1

Ім'я               Вид спорту         Результат  Категорія
-------------------------------------------------------------
Олена Ковальчук    Плавання               87.50  КМС
Максим Бондар      Легка атлетика         92.30  МС
Ірина Мельник      Плавання               78.40  I розряд
Андрій Шевченко    Бокс                   81.00  КМС
Софія Ткачук       Легка атлетика         95.10  МСМК

===== Облік спортивних результатів =====
1. Показати всі результати
2. Додати результат
3. Пошук за видом спорту
4. Найкращий результат
5. Середній результат
6. Рейтинг спортсменів
0. Вихід
Оберіть команду: 2

Ім'я спортсмена: Катерина Лисенко
Вид спорту: Плавання
Результат (бали): 120
Помилка: Результат має бути в діапазоні [0.0; 100.0].
Результат (бали): абв
Помилка: Результат 'абв' не є числом.
Результат (бали): 89,7
Категорія: майстер
Помилка: Невідома категорія 'майстер'. Допустимі: МСМК, МС, КМС, I розряд, II розряд, III розряд.
Категорія: кмс
Додано: Катерина Лисенко (89.70)
```

*Рисунок 2 — Виведення всіх результатів і додавання нового з перевіркою введених даних*

```
===== Облік спортивних результатів =====
1. Показати всі результати
2. Додати результат
3. Пошук за видом спорту
4. Найкращий результат
5. Середній результат
6. Рейтинг спортсменів
0. Вихід
Оберіть команду: 0

До побачення!
```

*Рисунок 3 — Завершення роботи програми*

## Пошук за видом спорту, найкращий і середній результат, рейтинг

```
(.venv) PS C:\Users\dasha\OneDrive\Рабочий стол\python_lab1> python -m sport_results.main

===== Облік спортивних результатів =====
1. Показати всі результати
2. Додати результат
3. Пошук за видом спорту
4. Найкращий результат
5. Середній результат
6. Рейтинг спортсменів
0. Вихід
Оберіть команду: 3

Вид спорту: плавання
Ім'я                Вид спорту         Результат  Категорія
------------------------------------------------------------
Олена Ковальчук     Плавання               87.50  КМС
Ірина Мельник       Плавання               78.40  I розряд

===== Облік спортивних результатів =====
1. Показати всі результати
2. Додати результат
3. Пошук за видом спорту
4. Найкращий результат
5. Середній результат
6. Рейтинг спортсменів
0. Вихід
Оберіть команду: 4

Найкращий результат: Софія Ткачук — Легка атлетика, 95.10 (МСМК)

===== Облік спортивних результатів =====
1. Показати всі результати
2. Додати результат
3. Пошук за видом спорту
4. Найкращий результат
5. Середній результат
6. Рейтинг спортсменів
0. Вихід
Оберіть команду: 5

Середній результат: 86.86
```

*Рисунок 4 — Запуск як module, пошук за видом спорту та найкращий результат*

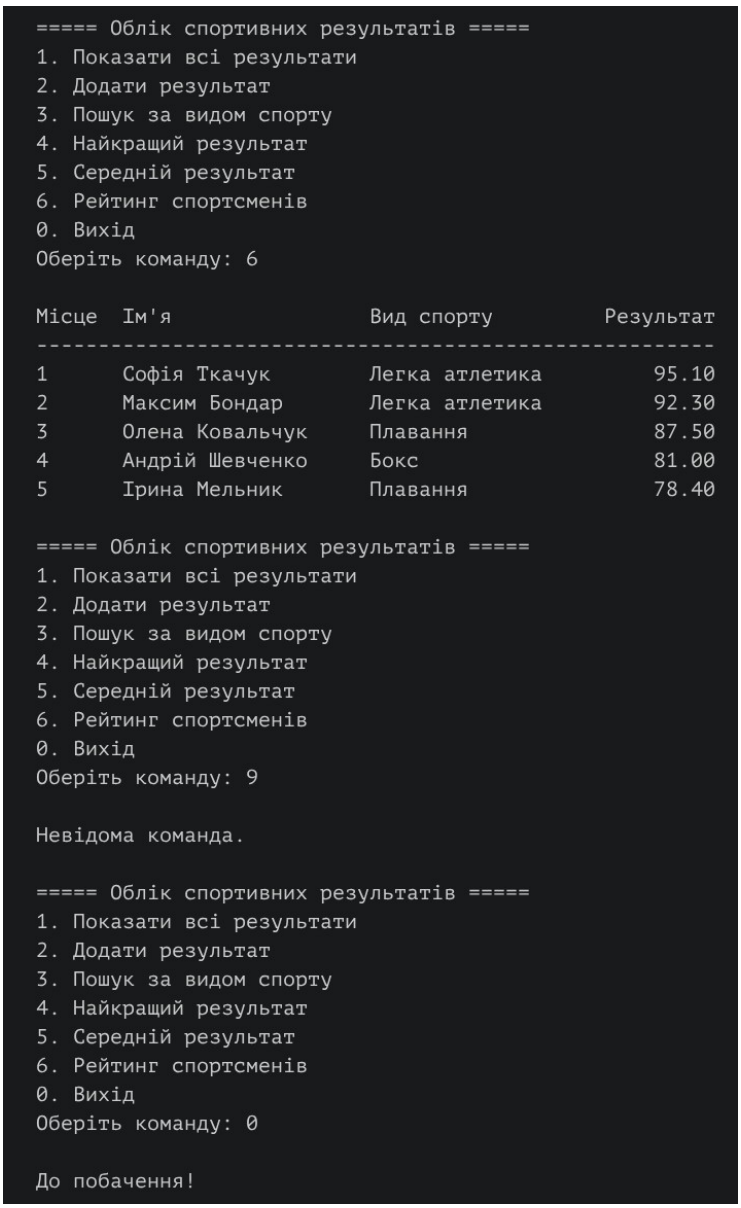



*Рисунок 5 — Середній результат, рейтинг та обробка невідомої команди*

## Валідація імені та оновлений рейтинг

```
(.venv) PS C:\Users\dasha\OneDrive\Рабочий стол\python_lab1> sport-results

===== Облік спортивних результатів =====
1. Показати всі результати
2. Додати результат
3. Пошук за видом спорту
4. Найкращий результат
5. Середній результат
6. Рейтинг спортсменів
0. Вихід
Оберіть команду: 2

Ім'я спортсмена:
Помилка: Поле 'ім'я' не може бути порожнім.
Ім'я спортсмена: Ольга 1
Помилка: Поле 'ім'я' не повинно містити цифр.
Ім'я спортсмена: Ольга Петренко
Вид спорту: Бокс
Результат (бали): 75
Категорія: II розряд
Додано: Ольга Петренко (75.00)

===== Облік спортивних результатів =====
1. Показати всі результати
2. Додати результат
3. Пошук за видом спорту
4. Найкращий результат
5. Середній результат
6. Рейтинг спортсменів
0. Вихід
Оберіть команду: 6

Місце  Ім'я               Вид спорту         Результат
-------------------------------------------------------
1      Софія Ткачук        Легка атлетика         95.10
2      Максим Бондар       Легка атлетика         92.30
3      Олена Ковальчук     Плавання               87.50
4      Андрій Шевченко     Бокс                   81.00
5      Ірина Мельник       Плавання               78.40
6      Ольга Петренко      Бокс                   75.00
```

*Рисунок 6 — Помилки валідації імені та формування рейтингу*

```
===== Облік спортивних результатів =====
1. Показати всі результати
2. Додати результат
3. Пошук за видом спорту
4. Найкращий результат
5. Середній результат
6. Рейтинг спортсменів
0. Вихід
Оберіть команду: 0

До побачення!
```

*Рисунок 7 — Вихід із програми*

### Запуск автоматизованих тестів

```
(.venv) PS C:\Users\dasha\OneDrive\Рабочий стол\python_lab1> python -m pytest -v
============================= test session starts ==============================
platform win32 -- Python 3.13.16, pytest-9.1.1, pluggy-1.7.0 -- C:\Users\dasha\OneDrive\Рабочий стол\python_lab1\.venv\Scripts\python.exe
rootdir: C:\Users\dasha\OneDrive\Рабочий стол\python_lab1
configfile: pyproject.toml
collecting ... collected 11 items

tests/test_services.py::test_add_result PASSED                           [  9%]
tests/test_services.py::test_find_by_sport_ignores_case PASSED           [ 18%]
tests/test_services.py::test_find_best_result PASSED                     [ 27%]
tests/test_services.py::test_find_best_result_empty PASSED               [ 36%]
tests/test_services.py::test_calculate_average_result PASSED             [ 45%]
tests/test_services.py::test_build_rating_order PASSED                   [ 54%]
tests/test_services.py::test_validate_result_errors[abc] PASSED          [ 63%]
tests/test_services.py::test_validate_result_errors[-1] PASSED           [ 72%]
tests/test_services.py::test_validate_result_errors[100.5] PASSED        [ 81%]
tests/test_services.py::test_validate_result_errors[] PASSED             [ 90%]
tests/test_services.py::test_validate_category_normalizes PASSED         [100%]

============================== 11 passed in 0.04s ==============================
```

*Рисунок 8 — Результат виконання тестів pytest*

## Результат git status

```
(.venv) PS C:\Users\dasha\OneDrive\Рабочий стол\python_lab1> git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

*Рисунок 9 — Результат git status*

## Результат git log --oneline

```
(.venv) PS C:\Users\dasha\OneDrive\Рабочий стол\python_lab1> git log --oneline
f03a7bb (HEAD -> main, origin/main) Move project to repository root
03cfd75 Add README with usage example
27f6d66 Add tests for services and validators
613fe8f Add menu-driven application entry point
f639e56 Add business logic and input validation
ec6d8cd Add Athlete model and custom exceptions
c99dca7 Create project structure
```

*Рисунок 10 — Результат git log --oneline*

## Відповіді на контрольні запитання

### 1. Що таке Python-проєкт?

Структурований набір вихідного коду, модулів, пакетів, залежностей, тестів, документації та конфігураційних файлів, організований для зручної розробки, запуску, тестування й поширення програми.

### 2. Чим проєкт відрізняється від одного Python script?

Скрипт — це один файл, що виконує конкретну задачу. Проєкт має модульну архітектуру, ізольоване середовище, опис залежностей, документацію, тести та історію змін у Git.

**3. Що таке Python module?**

Файл із розширенням .py, що містить функції, класи та змінні, які можна імпортувати в інші модулі.

**4. Що таке Python package?**

Каталог, який містить модулі та файл __init__.py і утворює окремий простір імен.

**5. Для чого потрібний __init__.py?**

Позначає каталог як пакет, виконується під час імпорту пакета, може містити його версію та визначати публічний інтерфейс.

**6. Що таке src-layout?**

Структура проєкту, у якій пакети з вихідним кодом розміщено в окремому каталозі src/, а не в корені репозиторію.

**7. Які переваги має src-layout?**

Унеможливлює випадковий імпорт коду з робочого каталогу замість встановленого пакета, змушує встановлювати пакет і тестувати саме його, чітко відокремлює код від конфігурації та тестів.

**8. Для чого використовується virtual environment?**

Для ізоляції інтерпретатора та бібліотек одного проєкту від інших проєктів і глобальної системи, щоб уникнути конфліктів версій.

**9. Як створити .venv?**

Командою python -m venv .venv у кореневому каталозі проєкту.

**10. Як активувати virtual environment?**

Windows: .venv\Scripts\activate; Linux/macOS: source .venv/bin/activate.

**11. Для чого використовується pip?**

Для встановлення, оновлення та видалення пакетів Python з PyPI або з локального каталогу.

**12. Чому рекомендується python -m pip?**

Так pip гарантовано запускається тим самим інтерпретатором Python (наприклад, з активного .venv), у який потрібно встановити пакет.

**13. Для чого потрібний pyproject.toml?**

Це стандартний конфігураційний файл проєкту, який описує систему збирання, метадані пакета, залежності, консольні команди та налаштування інструментів.

**14. Що містить секція [project]?**

Метадані пакета: name, version, description, readme, authors, requires-python, dependencies тощо.

**15. Що таке editable installation?**

Встановлення пакета командою pip install -e ., за якого в site-packages додається посилання на каталог розробки, тож зміни в коді діють без повторного встановлення.

**16. Для чого використовується if __name__ == "__main__":?**

Щоб код запускався лише тоді, коли файл виконується безпосередньо, а не під час імпорту модуля.

**17. Що таке Git?**

Розподілена система контролю версій для збереження історії змін файлів і спільної роботи над кодом.

**18. Що таке repository?**

Сховище проєкту під керуванням Git, яке містить файли та всю історію комітів, гілок і тегів (каталог .git).

**19. Що таке commit?**

Зафіксований знімок стану проєкту з унікальним хешем, автором, датою та повідомленням про зміни.

**20. Для чого використовується git status?**

Показує поточну гілку, змінені, нові (untracked) та підготовлені до коміту файли, а також стан відносно віддаленої гілки.

**21. Для чого git add?**

Додає зміни у вказаних файлах до staging area, тобто готує їх до наступного коміту.

**22. Для чого git commit?**

Зберігає підготовлені зміни зі staging area в історію репозиторію як новий коміт.

**23. Що показує git log?**

Історію комітів: хеш, автора, дату та повідомлення; з ключем --oneline — скорочений хеш і заголовок у один рядок.

**24. Для чого потрібен .gitignore?**

Задає шаблони файлів і каталогів, які Git не повинен відстежувати (середовище, кеш, артефакти збирання, секрети).

**25. Чому .venv не потрібно комітити?**

Каталог великий, залежить від платформи та шляхів конкретного комп'ютера і легко відтворюється командами python -m venv та pip install.

**26. Для чого потрібний README?**

Це основна документація проєкту: опис призначення, вимоги, встановлення, запуск, структура та автор.

**27. Що таке type hints?**

Анотації типів параметрів, значень, що повертаються, та змінних, які покращують читабельність коду та дозволяють статичну перевірку (mypy).

**28. Для чого програму розділяють на modules?**

Для поділу відповідальності, кращої читабельності, повторного використання коду та спрощення тестування й супроводу.

**29. Що таке business logic?**

Частина програми, що реалізує правила предметної області та обробку даних, — у цьому проєкті пошук, обчислення середнього, найкращого результату та рейтингу.

**30. Чому потрібно відокремлювати application entry point від business logic?**

Щоб бізнес-логіку можна було повторно використовувати й тестувати незалежно від способу введення/виведення (консоль, GUI, веб-API).

## Висновок

У ході лабораторної роботи було створено структурований Python-проєкт sport-results за принципом src-layout для обліку спортивних результатів (варіант 9). Налаштовано ізольоване віртуальне середовище, розроблено модель Athlete на базі @dataclass та окремий модуль бізнес-логіки з анотаціями типів, що реалізує додавання результату, пошук за видом спорту, визначення найкращого та середнього результату й формування рейтингу. Як завдання підвищеної складності реалізовано menu-driven interface з введенням даних із клавіатури та валідацію введення з власними винятками. Проєкт сконфігуровано через pyproject.toml з підтримкою editable installation та консольної команди, задокументовано у README.md, захищено від зайвих файлів за допомогою .gitignore, покрито тестами pytest і збережено в Git у вигляді логічних комітів із публікацією на GitHub.